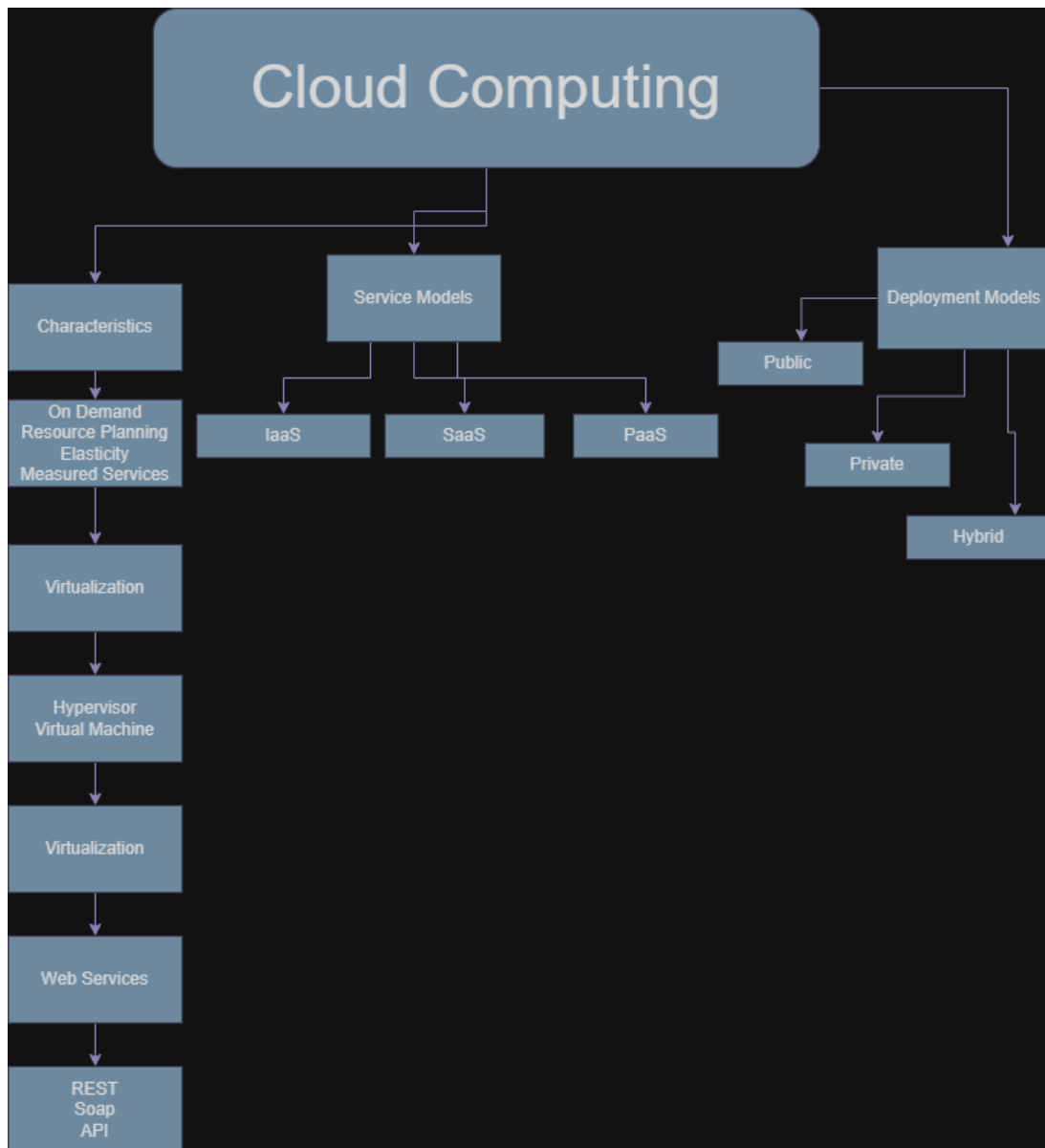
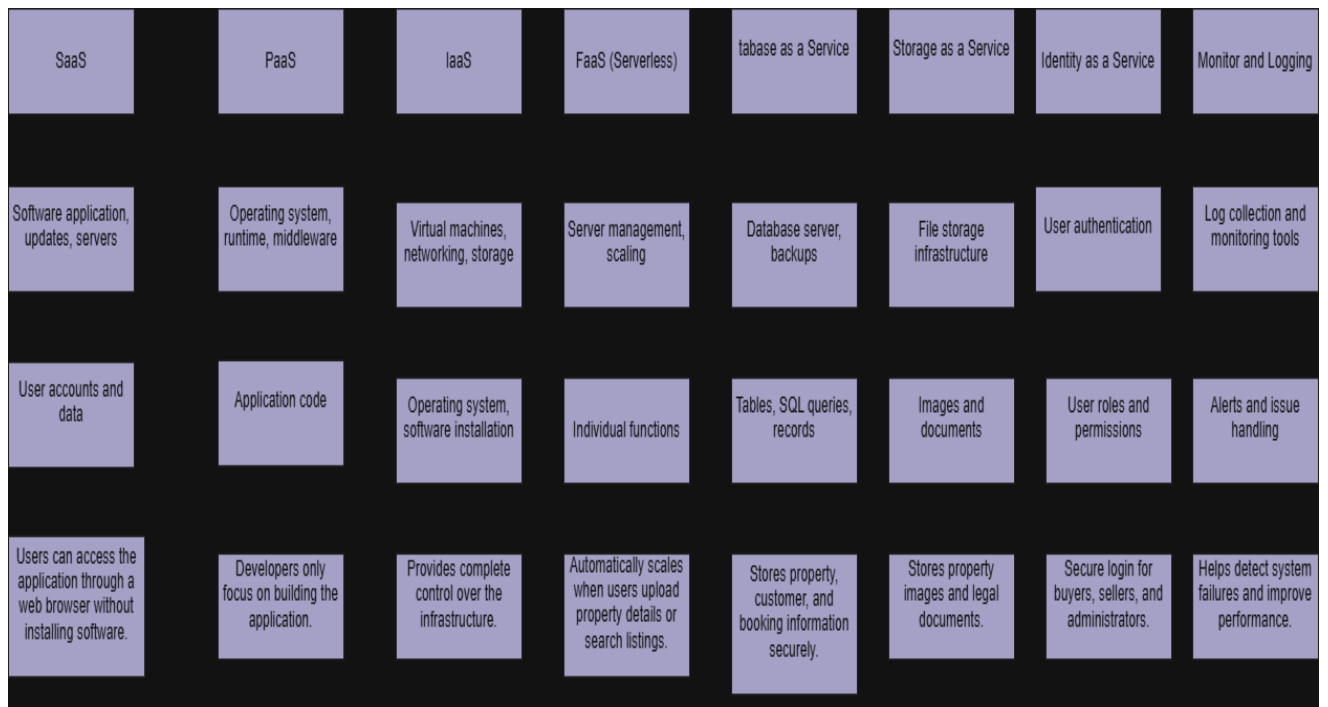


1.



2A.



2B.

Project Module	Selected Cloud Service	Why this Service?	Justification
Authentication	Firebase Authentication	Secure login and user management	Supports email/password authentication, Google Sign-In, and secure identity management.
Frontend Hosting	Netlify	Hosts the web application	Easy deployment with automatic HTTPS, CDN, and continuous deployment from GitHub.
Backend / API Layer	Render	Runs backend REST APIs	Provides reliable backend hosting with automatic deployment and scaling.

Project Module	Selected Cloud Service	Why this Service?	Justification
Database	Supabase PostgreSQL	Stores application data	Manages users, properties, bookings, and transactions securely.
Object Storage	Supabase Storage	Stores images and documents	Stores property photos, legal documents, and uploaded files securely.
Analytics / Logging	Google Analytics	Tracks website usage	Helps analyze visitor behavior and application performance.
Notification	Firebase Cloud Messaging (FCM)	Sends notifications	Sends booking confirmations, approval messages, and alerts to users.
AI / Python Module	Python Flask API	Property recommendation system	Executes AI algorithms separately from the main application.
Security	HTTPS + JWT Authentication	Secures communication	Encrypts data and protects APIs from unauthorized access.
CI/CD	GitHub Actions	Automates deployment	Automatically builds and deploys the application after every code update.
Backup	Supabase Automated Backup	Protects application data	Enables recovery of important data in case of failures.
Monitoring	Google Cloud Monitoring	Tracks application health	Monitors server performance, uptime, and resource usage.

Justification:

The selected cloud services are based on the functional requirements of the Property Buying & Rental Portal. Firebase Authentication provides secure user authentication, Netlify hosts the frontend efficiently, Render executes backend APIs, and Supabase manages the database and file storage. Google Analytics helps monitor user activity, while GitHub Actions automates deployment. HTTPS and JWT ensure secure communication, and monitoring and backup services improve reliability, scalability, and availability.

3.



4. Technical Justification

The concept map, service responsibility matrix, cloud service decision matrix, and architecture diagram demonstrate **Course Outcome 1 (CO1)** by showing the practical application of cloud computing concepts. The project uses the **SaaS service model** for delivering the application to users, while **PaaS** and **managed cloud services** simplify application deployment and maintenance. REST APIs enable communication between the frontend, backend, database, and storage services. Elasticity and scalability allow the system to handle varying user traffic by automatically adjusting computing resources. Managed compute services reduce infrastructure management through virtualization, allowing the cloud provider to maintain hardware and runtime environments. The responsibility matrices clearly define the shared responsibilities between the cloud provider and the application development team, ensuring security, reliability, and efficient resource utilization.

1. Effectiveness of the Concept Map

The concept map effectively represents the major principles of cloud computing by connecting cloud characteristics, service models, deployment models, virtualization, web services, APIs, scalability, and the project architecture. The hierarchical structure and labelled relationships clearly show how different cloud components interact. The inclusion of REST APIs, virtualization, and service models directly supports the design of the Property Buying & Rental Portal by illustrating how cloud technologies work together to provide a scalable and reliable solution.

2. Scalability, Elasticity, and Resource Management

Scalability allows the application to support increasing numbers of users by adding additional computing resources when required. Elasticity automatically allocates or releases resources depending on the workload, improving cost efficiency. Resource management ensures that compute, storage, and database resources are used effectively without unnecessary waste. Without these capabilities, the system could experience slow response times, service interruptions, higher operational costs, and poor user experience during periods of high demand.

=

3. Justification of APIs, Web Services, and Cloud Services

The selected cloud services were chosen based on the functional requirements of the project. REST APIs enable secure communication between the frontend, backend, database, and storage components. Firebase Authentication provides secure user

login, Supabase stores application data and files, Render hosts backend services, and GitHub Actions automates deployment. These cloud services improve system performance, simplify maintenance, enhance security, and reduce development complexity while ensuring high availability and scalability.

4. Relationships Between Components

Each component in the architecture performs a specific function while working together to deliver a reliable cloud application. The frontend communicates with the backend through REST APIs, the backend processes requests and stores information in the database, while object storage manages images and documents. Authentication verifies users before granting access, and monitoring continuously tracks system health. These interactions improve reliability by reducing failures, enhance performance through efficient request processing, and provide a smooth and secure user experience.

5. Reflection

Developing the concept map improved my understanding of cloud computing architecture by showing how service models, virtualization, APIs, storage, databases, and security are integrated into a single solution. It also helped me understand the shared responsibilities between cloud providers and application developers. If I were redesigning the architecture, I would include additional security mechanisms such as Web Application Firewalls (WAF), load balancing across multiple regions, AI-based monitoring, and disaster recovery strategies to further improve scalability, reliability, and fault tolerance.